CODE INVENTORY

DATA STORAGE (IN MEMORY)

The system does not read or write any files. All data is kept in memory while the program runs, in the three structures below, and starts fresh every time the program is opened.

NAME	Type	Description
`pendingQueue`	OrderQueue (custom linked list queue)	Orders waiting to be served, kept in First-In, First-Out order.
`servedOrders`	ArrayList<Order>	History of every served order, in the order they were served.
`menu`	ArrayList<MenuItem>	The 7 canteen items with their numbers and prices, loaded at start-up.

Table 1: In-Memory Data Storage

SOURCE FILES & CLASSES (6 Classes)

NAME	Role	Description
Main.java	User interface	Runs the console program: shows the menu, reads and validates input, calls OrderManager, and prints receipts and tables.
OrderManager.java	Business logic	Holds the pending queue, served history and menu, and runs the algorithms: enqueue, dequeue, linear search and insertion sort.
OrderQueue.java	Data structure	Hand-made First-In, First-Out queue built as a singly linked list with front, rear and size.
OrderNode.java	Data structure	One node of the linked list: an order plus a pointer to the next node.
Order.java	Data (model)	One customer order: ID, customer name, item, quantity, total, status and timestamp.
MenuItem.java	Data (model)	One item on the canteen menu: number, name and price.

Table 2: Source Files and Classes

CONTROL STRUCTURES

Control Structure	Aspect of Problem Where Used
Sequential	Statements run in order inside every method: creating the Scanner and OrderManager at start-up, loading the 7 menu items in `initializeMenu()`, computing total = price × quantity and building each order in `placeOrder()`, printing headers, receipts and tables, and reading the clock before and after the search and sort.
switch (arrow form)	Routing the main menu choice 1 to 7 in `main()` to place order, serve order, search, view pending, view completed, view menu, or exit.
if	Exit confirmation and the "Press ENTER" pause in `main()`, Windows-only console setup, printing a receipt only when an order was created, the closed-input check in `readLine()`, matching IDs in both linear searches, and resetting `rear` when the queue becomes empty.
if / else	Choosing the name error message (blank or no letters), showing the found order or "Order not found", choosing the number error message in `readIntChoice()`, choosing highest-first or lowest-first in insertion sort, and adding to an empty or non-empty queue in `enqueue()`.
Nested if	`readIntChoice()` first accepts digits only, then checks that the number is inside the allowed range.
if with early return (guard)	Stopping a feature when there is nothing to do: empty queue when serving or viewing pending orders, a cancelled serve (N), no served orders to sort, invalid input in `placeOrder()`, and empty-queue checks in `dequeue()` and `peek()`.
Logical operators (&&, ||)	Rejecting an order with a bad name, unknown item, or quantity outside 1 to 10; checking for `null` before the regex; checking the number range; and keeping the index at 0 or above in insertion sort.
Ternary operator (? :)	Choosing the first table heading: "Pos" for pending orders or "No." for completed orders in `printOrderTable()`.
while loop (flag-controlled)	The main program loop keeps showing the menu until `running` is set to false by a confirmed exit.
while loop (condition-controlled)	Asking for the customer name again until it has a letter, walking the linked list until `current` is null in `getAllOrders()`, and shifting orders in insertion sort.
while (true) loop	Repeating an input prompt until the input is valid: a whole number in range in `readIntChoice()` and a Y or N answer in `confirm()`. The loop ends with `return`.
Standard for loop	Outer loop of insertion sort, taking each order from index 1 to n − 1 as the key.
Enhanced for-each loop	Printing the menu, printing each row of the order tables, linear search through the menu, and linear search through the pending orders and then the served orders.
Nested loops	Insertion sort: a `while` loop inside the `for` loop shifts orders to open a space for the key.

Table 3: Control Structures (continued)

Control Structure	Aspect of Problem Where Used
try-catch	Keeping the program running if the Windows UTF-8 console command fails, and treating numbers too large for an `int` as invalid input instead of crashing.
return	Leaving a method as soon as the result is known: first match in a linear search, a valid number, a Y or N answer, and early exits when a feature has nothing to do.
System.exit(0)	Ending the program cleanly when the input is closed (Ctrl+Z or Ctrl+D) in `readLine()`.
Not used	`do-while`, `break`, `continue`, `throw`, `finally` and recursion are not used. Loops end through their condition, the `running` flag, or `return`.

Table 3: Control Structures

CONTROL STRUCTURE LOCATIONS

Every control structure in the source code, with its file, line number and method.

Control Structure	Location	Purpose
while (flag)	Main.java:31 — main()	Main program loop that shows the menu again after every feature.
switch	Main.java:46 — main()	Runs the feature for choices 1 to 7. No default case is needed because only 1 to 7 is accepted.
if	Main.java:54 — main()	Ends the program only if the user confirms Exit.
if	Main.java:61 — main()	Shows "Press ENTER to return to main menu..." while still running.
if	Main.java:74 — enableUtf8Output()	Runs the code-page command only on Windows.
try-catch	Main.java:75 — enableUtf8Output()	Ignores a failed console command so the program keeps running.
while	Main.java:104 — handlePlaceOrder()	Asks for the name again until it contains a letter.
if / else	Main.java:105 — handlePlaceOrder()	Shows "cannot be empty" or "must contain letters".
for-each	Main.java:115 — handlePlaceOrder()	Prints each menu item for the customer to choose.
if	Main.java:126 — handlePlaceOrder()	Prints the receipt only if the order was created.
if + return	Main.java:148 — handleServeNextOrder()	Stops when the queue is empty.

Table 4: Control Structure Locations (continued)

Control Structure	Location	Purpose
if + return	Main.java:156 — handleServeNextOrder()	Cancels the serve when the user answers N.
if / else	Main.java:187 — handleSearchOrder()	Shows "Order not found" or the order details and search time.
if + return	Main.java:211 — handleViewPendingOrders()	Shows a message when no orders are pending.
if + return	Main.java:226 — handleViewCompletedOrdersSorted()	Shows "No served orders yet." instead of sorting.
for-each	Main.java:249 — handleViewMenu()	Prints each menu item as a table row.
ternary (? :)	Main.java:258 — printOrderTable()	Uses "Pos" for pending or "No." for completed orders.
for-each	Main.java:261 — printOrderTable()	Prints one numbered row per order.
if (two)	Main.java:284–285 — truncate()	Handles null text and text that already fits.
if	Main.java:293 — readLine()	Detects closed input.
System.exit(0)	Main.java:295 — readLine()	Ends the program when input is closed.
while (true)	Main.java:305 — readIntChoice()	Repeats until a valid number is typed.
if (nested)	Main.java:307, 310 — readIntChoice()	Accepts digits only, then checks the range.
try-catch	Main.java:308 — readIntChoice()	Catches numbers too large for an int.
if / else	Main.java:317 — readIntChoice()	Chooses the error message to show.
while (true)	Main.java:327 — confirm()	Repeats until the answer is Y or N.
if (two)	Main.java:330–331 — confirm()	Returns true for Y and false for N.
for-each	OrderManager.java:50 — getMenuItemById()	Linear search through the menu.
if	OrderManager.java:51 — getMenuItemById()	Returns the item whose ID matches.
if (||)	OrderManager.java:74 — placeOrder()	Rejects a bad name, unknown item, or quantity outside 1 to 10.
if	OrderManager.java:105 — serveNextOrder()	Marks and saves the order only if one was dequeued.

Table 4: Control Structure Locations (continued)

Control Structure	Location	Purpose
for-each (two)	OrderManager.java:124, 131 — linearSearchOrder()	Linear search through pending orders, then served orders.
if (two)	OrderManager.java:125, 132 — linearSearchOrder()	Returns the order whose ID matches.
for	OrderManager.java:161 — getServedOrdersSortedByAmount()	Insertion sort outer loop (i = 1 to n − 1).
if / else	OrderManager.java:165 — getServedOrdersSortedByAmount()	Chooses highest-first or lowest-first.
while (two)	OrderManager.java:167, 173 — getServedOrdersSortedByAmount()	Insertion sort inner loop that shifts orders.
if / else	OrderQueue.java:29 — enqueue()	Adds to an empty queue or links after the rear.
if + return	OrderQueue.java:44 — dequeue()	Returns null when the queue is empty.
if	OrderQueue.java:49 — dequeue()	Clears rear when the queue becomes empty.
if + return	OrderQueue.java:61 — peek()	Returns null when the queue is empty.
while	OrderQueue.java:89 — getAllOrders()	Walks the linked list until current is null.

Table 4: Control Structure Locations

LIBRARIES AND PACKAGES

Library / Class	Package	Used For
Scanner	java.util	Reading the user's keyboard input line by line.
List	java.util	Type of the menu, the served history and every list of orders.
ArrayList	java.util	Dynamic arrays for the menu, served history, sorting copy and traversal results.
LocalDateTime	java.time	Getting the date and time an order is placed.
DateTimeFormatter	java.time.format	Formatting timestamps as yyyy-MM-dd HH:mm:ss.
PrintStream	java.io	UTF-8 console output so the peso sign (₱) prints correctly.
FileOutputStream	java.io	Connecting the UTF-8 output stream to the console.

Library / Class	Package	Used For
System	java.lang	Console output and input, OS name, nanoTime() timing, setOut() and exit().
String	java.lang	format, valueOf, trim, isEmpty, matches (regex), equalsIgnoreCase, substring, length, startsWith, repeat.
Integer	java.lang	parseInt() to read numbers and MAX_VALUE as the "no upper limit" for Order IDs.
ProcessBuilder	java.lang	Running the Windows command chcp 65001 to switch the console to UTF-8.
Process	java.lang	start() and waitFor() for that command.
Exception	java.lang	Catching a failed console command in enableUtf8Output().
NumberFormatException	java.lang	Catching numbers too large for an int in readIntChoice().
Object	java.lang	Parent of every class; toString() is overridden in Order and MenuItem.
Override	java.lang	Annotation that marks the toString() overrides.

Table 5: Libraries and Packages

The first nine classes are imported; the `java.lang` classes are available without an import. No external libraries are used, and the queue is hand-made: `java.util.Queue` and `java.util.LinkedList` are not used. The regular expressions are `".*\\p{L}.*"` (the name must contain a letter) and `"[0-9]+"` (digits only).

VARIABLES

User Interface Variables ***(Main)***

Variable (Type & Name)	Description
Scanner scanner	private static final. Reads the user's keyboard input.
OrderManager manager	private static final. The single OrderManager every menu option uses.
int RECEIPT_VALUE_WIDTH	private static final, value 36. Width of the value column in receipt boxes.
String RECEIPT_BORDER	private static final. The +-----+ line printed above and below each receipt.

Table 6: Main Class Variables

Business Logic Variables ***(OrderManager)***

Variable (Type & Name)	Description
int MIN_QUANTITY	public static final, value 1. Smallest quantity allowed per order

Variable (Type & Name)	Description
int MAX_QUANTITY	public static final, value 10. Largest quantity allowed per order.
OrderQueue pendingQueue	private final. First-In, First-Out queue of orders waiting to be served.
List<Order> servedOrders	private final. History of served orders, in the order they were served.
List<MenuItem> menu	private final. The 7 canteen menu items.
int nextOrderId	private. ID for the next order; starts at 1001 and increases by 1.
DateTimeFormatter timeFormatter	private final. Formats timestamps as yyyy-MM-dd HH:mm:ss.

Table 7: OrderManager Class Variables

Queue Variables ***(OrderQueue and OrderNode)***

Variable (Type & Name)	Description
OrderNode front	OrderQueue, private. First node: the next order to be served.
OrderNode rear	OrderQueue, private. Last node: the most recently placed order.
int size	OrderQueue, private. Number of orders waiting in the queue.
Order data	OrderNode, public. The order stored in this node.
OrderNode next	OrderNode, public. The next node in the chain (null for the last node).

Table 8: OrderQueue and OrderNode Variables

Data Variables ***(Order and MenuItem)***

Variable (Type & Name)	Description
int orderId	Order, private final. Unique order number (1001, 1002, ...).
String customerName	Order, private final. Name of the customer.
String itemName	Order, private final. Name of the ordered menu item.
int quantity	Order, private final. Number of servings (1 to 10).
double totalAmount	Order, private final. Price × quantity, in pesos.
String status	Order, private. "PENDING" when placed and "SERVED" when served; the only field that changes.
String timestamp	Order, private final. Date and time the order was placed.
int id	MenuItem, private final. Item number shown on the menu (1 to 7).
String name	MenuItem, private final. Name of the meal or drink.

Table 9: Order and MenuItem Variables (continued)

Variable (Type & Name)	Description
double price	MenuItem, private final. Price in pesos.

Table 9: Order and MenuItem Variables

Parameters and Local Variables ***(all methods)***

Variable (Type & Name)	Method	Description
String[] args	Main.main	Command-line arguments (not used).
boolean running	Main.main	Keeps the main loop going; false after a confirmed exit.
int choice	Main.main	Menu option typed by the user (1 to 7).
Exception e	Main.enableUtf8Output	Error from the console command (ignored on purpose).
String customerName	Main.handlePlaceOrder	Customer name typed by the user.
List<MenuItem> menu	Main.handlePlaceOrder	Menu items shown for the customer to choose.
MenuItem item	Main.handlePlaceOrder	Each menu item printed by the loop.
int itemChoice	Main.handlePlaceOrder	Menu item number chosen (1 to 7).
int quantity	Main.handlePlaceOrder	Quantity chosen (1 to 10).
Order newOrder	Main.handlePlaceOrder	The order created by OrderManager.
Order next	Main.handleServeNextOrder	The front order shown before confirming.
Order served	Main.handleServeNextOrder	The order that was dequeued and served.
int orderId	Main.handleSearchOrder	Order ID to search for.
long startTime, endTime	Main.handleSearchOrder	Clock readings before and after the search (nanoseconds).
Order found	Main.handleSearchOrder	The order found, or null.
double elapsedMicroseconds	Main.handleSearchOrder	Search time in microseconds.
List<Order> pending	Main.handleViewPendingOrders	Pending orders in FIFO order.
List<Order> completed	Main.handleViewCompletedOrdersSorted	Copy of the served orders.
long startTime, endTime	Main.handleViewCompletedOrdersSorted	Clock readings before and after the sort (nanoseconds).

Table 10: Parameters and Local Variables (continued)

Variable (Type & Name)	Method	Description
List<Order> sorted	Main.handleViewCompletedOrdersSorted	Served orders sorted highest amount first.
double elapsedMicroseconds	Main.handleViewCompletedOrdersSorted	Sort time in microseconds.
MenuItem item	Main.handleViewMenu	Each menu item printed as a row.
List<Order> list	Main.printOrderTable	Orders to print.
boolean isPending	Main.printOrderTable	True for the pending table, false for completed.
int index	Main.printOrderTable	Row number, increased for each order.
Order o	Main.printOrderTable	Each order printed by the loop.
String label, value	Main.printReceiptRow	Left and right text of one receipt row.
double amount	Main.peso	Amount to format as ₱0.00.
String text	Main.truncate	Text to shorten.
int max	Main.truncate	Column width the text must fit.
int min, max	Main.readIntChoice	Allowed range of the number.
String line	Main.readIntChoice	Text typed by the user.
int val	Main.readIntChoice	The typed text converted to a number.
NumberFormatException e	Main.readIntChoice	Error for numbers too large for an int.
String question	Main.confirm	The Y/N question to ask.
String answer	Main.confirm	The user's answer.
int id	OrderManager.getMenuItemById	Menu item number to find.
MenuItem item	OrderManager.getMenuItemById	Each menu item checked by the search.
String customerName	OrderManager.isValidCustomerName	Name to check for at least one letter.
String customerName	OrderManager.placeOrder	Customer name (trimmed before saving).
int menuItemId, quantity	OrderManager.placeOrder	Chosen item number and quantity.
MenuItem item	OrderManager.placeOrder	The chosen menu item.
double totalAmount	OrderManager.placeOrder	Price × quantity.
String timestamp	OrderManager.placeOrder	Current date and time as text.

Variable (Type & Name)	Method	Description
Order newOrder	OrderManager.placeOrder	The order created and enqueued.
Order order	OrderManager.serveNextOrder	The order taken from the front of the queue.
int orderId	OrderManager.linearSearchOrder	Order ID to search for.
List<Order> pendingList	OrderManager.linearSearchOrder	Pending orders from the queue.
Order order	OrderManager.linearSearchOrder	Each order compared (one loop variable per list).
boolean descending	OrderManager.getServedOrdersSortedByAmount	True sorts highest first, false lowest first.
List<Order> list	OrderManager.getServedOrdersSortedByAmount	The copy being sorted.
int n	OrderManager.getServedOrdersSortedByAmount	Number of served orders.
int i	OrderManager.getServedOrdersSortedByAmount	Outer loop index (position of the key).
Order keyOrder	OrderManager.getServedOrdersSortedByAmount	The order being inserted into the sorted part.
int j	OrderManager.getServedOrdersSortedByAmount	Inner loop index that moves left.
Order order	OrderQueue.enqueue	Order to add at the rear.
OrderNode newNode	OrderQueue.enqueue	New node that holds the order.
Order removedOrder	OrderQueue.dequeue	The front order being removed and returned.
List<Order> orders	OrderQueue.getAllOrders	List that collects the orders.
OrderNode current	OrderQueue.getAllOrders	Node currently being visited.
Order data	OrderNode constructor	The order the new node will hold.
orderId, customerName, itemName, quantity, totalAmount, status, timestamp	Order constructor	Values copied into the new order's fields.
String status	Order.setStatus	The new status ("SERVED").
int id, String name, double price	MenuItem constructor	Values copied into the new menu item's fields.

Table 10: Parameters and Local Variables

FUNCTIONS (METHODS)

User Interface Functions ***(Main)***. All are static.

Function	Description	Called From
main(String[] args) : void	Program entry point. Sets up UTF-8 output, then loops: header, menu, choice, feature, pause, until the user exits.	Java runtime
enableUtf8Output() : void	On Windows, runs chcp 65001 and replaces System.out with a UTF-8 stream so ₱ displays.	Main.java:29
printHeader() : void	Prints the title and the number of pending and served orders.	Main.java:32
handlePlaceOrder() : void	Option 1: reads the name, item and quantity, places the order, prints the receipt.	Main.java:47
handleServeNextOrder() : void	Option 2: shows the next order, asks Y/N, serves it, prints the receipt.	Main.java:48
handleSearchOrder() : void	Option 3: reads an Order ID, runs linear search, shows the result and time.	Main.java:49
handleViewPendingOrders() : void	Option 4: prints pending orders in FIFO order.	Main.java:50
handleViewCompletedOrdersSorted() : void	Option 5: sorts served orders highest amount first, prints them with the time.	Main.java:51
handleViewMenu() : void	Option 6: prints the menu as a table.	Main.java:52
printOrderTable(List<Order>, boolean) : void	Prints an order table ("Pos" or "No." column).	Main.java:216, 236
printReceiptRow(String, String) : void	Prints one aligned "label : value" receipt row.	21 calls in Main
peso(double) : String	Formats an amount as ₱0.00.	Main.java:133, 168, 196
truncate(String, int) : String	Shortens text that is too long and adds "…".	Main.java:265, 266, 276
readLine() : String	Reads one line; ends the program if input is closed.	5 calls in Main
readIntChoice(int, int) : int	Reads a whole number in a range, asking again until valid.	Main.java:43, 120, 123, 180
confirm(String) : boolean	Asks a Y/N question until the answer is Y or N.	Main.java:54, 156

Table 11: Main Functions

Business Logic Functions ***(OrderManager)***

Function	Description	Called From
OrderManager()	Constructor: creates the queue and lists, sets the first ID to 1001, loads the menu.	Main.java:22
initializeMenu() : void	Private. Adds the 7 menu items and prices.	OrderManager.java:29
getMenu() : List<MenuItem>	Returns the menu list.	Main.java:114, 249
getMenuItemById(int) : MenuItem	Linear search for a menu item by number; null if not found.	OrderManager.java:73
isValidCustomerName(String) : boolean	Static. True if the name contains at least one letter.	Main.java:104; OrderManager.java:74
placeOrder(String, int, int) : Order	Validates input, computes the total, creates the order and enqueues it.	Main.java:125
peekNextOrder() : Order	Returns the front order without removing it.	Main.java:147
serveNextOrder() : Order	Dequeues the front order, marks it SERVED, adds it to the history.	Main.java:161
linearSearchOrder(int) : Order	Linear search by Order ID: pending queue first, then served list.	Main.java:183
getPendingOrders() : List<Order>	Returns the pending orders in FIFO order.	Main.java:92, 135, 210
getServedOrdersSortedByAmount(boolean) : List<Order>	Insertion sort of a copy of the served orders by total amount.	Main.java:232
getServedOrders() : List<Order>	Returns a copy of the served history.	Main.java:92, 225

Table 12: OrderManager Functions

Queue Functions ***(OrderQueue and OrderNode)***

Function	Description	Called From
OrderQueue()	Constructor: empty queue (front and rear null, size 0).	OrderManager.java:24
enqueue(Order) : void	Adds an order at the rear. O(1).	OrderManager.java:84
dequeue() : Order	Removes and returns the front order, or null if empty. O(1).	OrderManager.java:104
peek() : Order	Returns the front order without removing it. O(1).	OrderManager.java:93
isEmpty() : boolean	True if the queue has no orders. O(1).	OrderQueue.java:44, 61
getSize() : int	Returns the number of orders. O(1).	Not called

Table 13: OrderQueue and OrderNode Functions (continued)

Function	Description	Called From
getAllOrders() : List<Order>	Traverses the linked list front to rear. O(n).	OrderManager.java:123, 145
OrderNode(Order)	Constructor: a node holding an order, with next = null.	OrderQueue.java:28

Table 13: OrderQueue and OrderNode Functions

Data Functions ***(Order and MenuItem)***

Function	Description	Called From
Order(int, String, String, int, double, String, String)	Constructor: creates an order with all its details.	OrderManager.java:83
getOrderId() : int	Returns the order ID.	Receipts, tables, linear search
getCustomerName() : String	Returns the customer name.	Receipts and tables
getItemName() : String	Returns the item name.	Receipts and tables
getQuantity() : int	Returns the quantity.	Receipts and tables
getTotalAmount() : double	Returns the total amount.	Receipts, tables, insertion sort
getStatus() : String	Returns PENDING or SERVED.	Receipts and tables
setStatus(String) : void	Changes the status (the only setter).	OrderManager.java:106
getTimestamp() : String	Returns when the order was placed.	Main.java:136, 198
toString() : String	Order: one-line summary. Overrides Object.toString().	Not called
MenuItem(int, String, double)	Constructor: creates a menu item.	OrderManager.java:36–42
getId() : int	Returns the item number.	Main.java:250; OrderManager.java:51
getName() : String	Returns the item name.	Main.java:250; OrderManager.java:83
getPrice() : double	Returns the price.	Main.java:250; OrderManager.java:80
toString() : String	MenuItem: "[1] Chicken Rice Meal - ₱85.00". Overrides Object.toString().	Main.java:116 (automatic)

Table 14: Order and MenuItem Functions

OBJECTS

Object	Class	Description
scanner	Scanner	Main.java:21. One object that reads keyboard input for the whole program.
manager	OrderManager	Main.java:22. The single controller used by every menu option.
pendingQueue	OrderQueue	OrderManager.java:24. Holds the orders waiting to be served.
servedOrders	ArrayList<Order>	OrderManager.java:25. Grows by one each time an order is served.
menu	ArrayList<MenuItem>	OrderManager.java:26. Holds the 7 menu items.
7 menu items	MenuItem	OrderManager.java:36–42. Chicken Rice Meal ₱85, Burger Steak w/ Rice ₱75, Pork Tonkatsu Meal ₱90, Spaghetti w/ Garlic Bread ₱65, Crispy French Fries ₱40, Canteen Iced Tea ₱25, Bottled Mineral Water ₱15.
timeFormatter	DateTimeFormatter	OrderManager.java:28. Created with ofPattern(); formats timestamps.
newOrder	Order	OrderManager.java:83. One per placed order (IDs 1001, 1002, ...). The same object moves from the queue to the served history.
newNode	OrderNode	OrderQueue.java:28. One per enqueued order; dropped (garbage-collected) when the order is dequeued.
list (sorting copy)	ArrayList<Order>	OrderManager.java:158. The copy that insertion sort rearranges.
served history copy	ArrayList<Order>	OrderManager.java:185. Safe copy returned by getServedOrders().
orders	ArrayList<Order>	OrderQueue.java:87. Pending orders collected by getAllOrders().
timestamp source	LocalDateTime	OrderManager.java:81. LocalDateTime.now() for each placed order, turned into text right away.
console command	ProcessBuilder, Process	Main.java:76–79. Runs chcp 65001 once at start-up (Windows only).
UTF-8 output	PrintStream, FileOutputStream	Main.java:84. Replaces System.out so ₱ prints correctly.

Table 15: Objects Created by the Program

PRINCIPLES OF OOP

Principle	How It Is Used in the Code
Classes and Objects	Six classes divide the problem into parts. At run time the program creates one OrderManager and one OrderQueue, seven MenuItem objects, one Order for each order placed, and one OrderNode for each order waiting. Each object keeps its own data; for example, every Order has its own ID, name and status.
Encapsulation	Fields are private and are read through getters. Most fields of Order and MenuItem are final, so they never change. The only changeable field, Order.status, is changed only through setStatus() when an order is served. OrderQueue hides front, rear and size so no other class can break the links. getServedOrders() returns a copy so the real history cannot be changed from outside.
Abstraction	Main calls placeOrder(), serveNextOrder(), linearSearchOrder() and getServedOrdersSortedByAmount() without knowing how the queue, search or sort work. OrderManager calls enqueue() and dequeue() without handling the front, rear or next pointers itself.
Inheritance	The code has no extends keyword, so there are no parent and child classes of its own. Every class automatically inherits from java.lang.Object, which provides toString(), equals() and hashCode(). Order and MenuItem use this by overriding toString().
Polymorphism	Order and MenuItem override toString() (marked @Override). At Main.java:116, " " + item makes Java call MenuItem's own toString() while the program runs, printing "[1] Chicken Rice Meal - ₱85.00". This is runtime polymorphism. There is no method overloading.
Composition and Aggregation	Classes are built from other objects ("has-a"). Main has an OrderManager; OrderManager has an OrderQueue, the MenuItems and the served Orders; OrderQueue has OrderNodes; each OrderNode has an Order. Objects a class creates and owns are composition. The served orders, which come from the queue, are aggregation.
Static Members	Everything in Main is static, because the program runs from main() without creating a Main object. MIN_QUANTITY, MAX_QUANTITY and isValidCustomerName() are static, so Main uses them through the class name, for example OrderManager.MAX_QUANTITY.
Constructors	Each class sets up its objects in a constructor: OrderManager builds the queue, lists and menu; OrderQueue starts empty; OrderNode starts with next = null; Order and MenuItem copy their values into final fields.
Generics	Typed collections such as List<Order> and List<MenuItem> make the compiler check that a list of orders only holds Order objects, so no casting is needed.
Separation of Concerns	Each class has one job: Main handles the screen, OrderManager the rules and algorithms, OrderQueue and OrderNode the pending orders, and Order and MenuItem the data. A change to one part does not affect the others.
Not Used	Interfaces, abstract classes, user-defined inheritance (extends) and method overloading are not used; this program does not need them.